

Checkpoint3: Planning and Exploration

Peilin Meng, Yijie Liao, Yun Jong Na, Yunzhu Deng
University of Michigan

I. A* PATH PLANNER

A. Overview

The A* path planner is implemented in `astar.cpp` as the class `AStarPlanner` within the `mbot_nav` namespace. Its role in the navigation stack is to compute a collision-free path from the robot's current pose to a given goal pose on the occupancy grid. The planner operates on an `ObstacleDistanceGrid`, which provides both occupancy values and a precomputed distance-to-nearest-obstacle field, enabling efficient safety-margin enforcement.

The class exposes a single public method, `planPath()`, which orchestrates the entire pipeline: coordinate conversion, validity checking, A* graph search, path reconstruction, and path smoothing. Internally it relies on four private helper functions. The call graph is shown in Figure 1.

B. Data Structures

The `AStarNode` struct is defined in `astar.hpp`. Each node contains four fields: grid coordinates (x, y) , a g -cost field `cost` representing the actual accumulated cost from the start, and a f -cost field `priority` equal to $g + h$. The struct overloads `operator<` to compare by `priority`, so that the standard-library priority queue can sort nodes in ascending f -cost order.

Three data structures in A* search are summarized in Table I.

TABLE I. Core data structures in the A* search.

Variable	Purpose
<code>open</code>	Frontier nodes sorted by f -cost
<code>cost_so_far</code>	Best known g -cost per visited cell
<code>came_from</code>	Parent pointer for path reconstruction

`open` is a `std::priority_queue` parameterized with `std::greater<>`, which invokes the `operator<` defined on `AStarNode` and turns the default max-heap into a min-heap, so the node with the smallest f -cost is always popped first. `cost_so_far` is an `std::unordered_map<int, float>` that maps a cell's one-dimensional index to its best known g -cost. `came_from` is an `std::unordered_map<int, int>` that maps each cell index to the index of its predecessor on the cheapest known path, enabling backward traceback from the goal to the start.

Grid cells are identified by a one-dimensional index computed as $\text{idx} = y \times \text{width} + x$ via the inline helper `toIndex()`.

C. Implementation Detail

1) **planPath()**: `planPath()` receives an `Obstacle Distance Grid`, a start pose, a goal pose, and an output path. Its execution proceeds in six stages:

a) *Stage 1: Coordinate Conversion.*: World-frame coordinates (x_w, y_w) are converted to grid indices via

$$x_g = \left\lfloor \frac{x_w - o_x}{r} \right\rfloor, \quad y_g = \left\lfloor \frac{y_w - o_y}{r} \right\rfloor,$$

where o_x, o_y are the map origin coordinates and r is the grid resolution in meters per cell.

b) *Stage 2: Validity Checks.*: The function verifies that both the start and goal cells lie within the grid boundaries and that their occupancy values are below 50 (i.e., not inside a known obstacle). If either check fails, the function returns `false`.

c) *Stage 3: Initialization.*: The open list, `cost_so_far`, and `came_from` are initialized. The start node is pushed with $g = 0$ and $f = h(\text{start}, \text{goal})$. The start node's parent is set to itself, serving as the termination sentinel during reconstruction.

d) *Stage 4: A* Search Loop.*: In each iteration the node with the lowest f -cost is popped. If it is the goal, the search terminates and proceeds to path extraction (Stage 5). Otherwise, its eight neighbors (four cardinal and four diagonal) are examined. For each neighbor:

- 1) **Obstacle filter**: cells with occupancy ≥ 50 are skipped.
- 2) **Clearance filter**: cells whose distance to the nearest obstacle is less than a threshold $d_{\min} = 0.20$ m are skipped, ensuring the robot maintains a safe margin from walls.
- 3) **Penalty computation**: the cost penalty p is accumulated from two sources:
 - *Unknown-cell penalty*: if the cell's occupancy equals -1 (unknown), a flat penalty $w_u = 2.0$ is added.
 - *Proximity penalty*: an inverse-distance term $w_c / (d_{\text{obs}} + 0.10)$ with $w_c = 1.0$ discourages paths near obstacles.
- 4) **Move cost**: $\|\Delta\|_2$, equal to 1 for cardinal moves and $\sqrt{2}$ for diagonal moves.
- 5) **Relaxation**: if the tentative cost $g' = g_{\text{curr}} + \|\Delta\|_2 + p$ is lower than the best known cost for the neighbor, or if the neighbor has not yet been visited, update `cost_so_far` and `came_from`, and push the neighbor with priority $f = g' + h$.

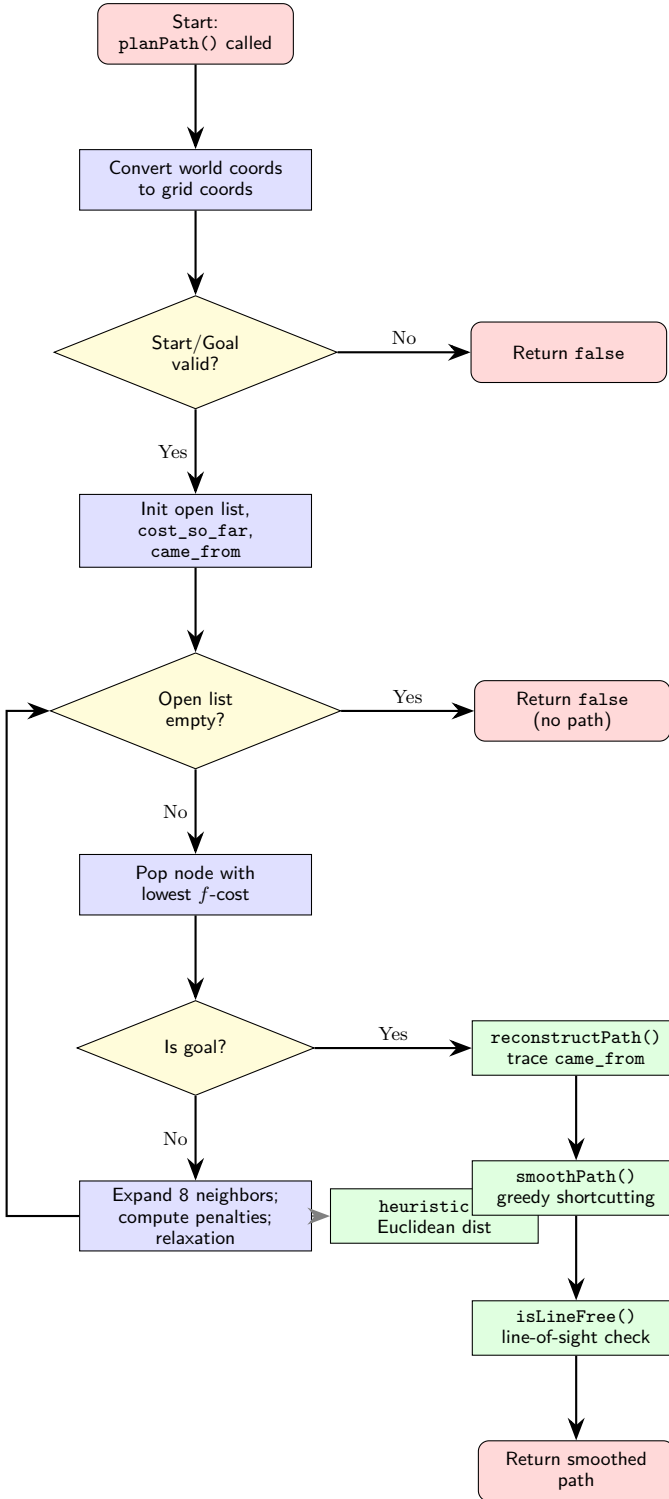


Fig. 1. Call graph and control flow of the A* path planner. Solid arrows show control flow; the dashed arrow indicates that `heuristic()` is invoked during neighbor expansion.

The total edge cost from a node n to its neighbor n' is therefore

$$c(n, n') = \underbrace{\|\Delta\|_2}_{\text{move cost}} + \underbrace{\frac{w_c}{d_{\text{obs}}(n') + 0.10}}_{\text{proximity penalty}} + \underbrace{w_u \cdot \mathbf{1}[\text{unknown}(n')]}_{\text{unknown penalty}}.$$

e) *Stage 5: Path Extraction.*: Upon reaching the goal, `reconstructPath()` is called to trace the parent chain. Grid coordinates are converted back to world coordinates, and a heading θ_i is assigned to each waypoint:

$$\theta_i = \begin{cases} \arctan(y_{i+1} - y_i, x_{i+1} - x_i), & i < N - 1, \\ \theta_{\text{goal}}, & i = N - 1. \end{cases}$$

f) *Stage 6: Smoothing.*: The raw path is passed to `smoothPath()` for simplification before being returned.

2) *heuristic()* — *Admissible Heuristic*: The heuristic function returns the Euclidean distance between two grid cells:

$$h(n, n_{\text{goal}}) = \sqrt{(\Delta x)^2 + (\Delta y)^2}.$$

Since Euclidean distance never overestimates the true shortest-path distance on an 8-connected grid, this heuristic is *admissible*, guaranteeing A* optimality (with respect to the cost function in Eq. I-C1d).

3) *reconstructPath()* — *Parent-Chain Traceback*: Starting from the goal index, this function iteratively looks up each node's parent in the `came_from` map, converting one-dimensional indices back to (x, y) pairs via $x = \text{idx} \bmod w$ and $y = \lfloor \text{idx}/w \rfloor$. The loop terminates when a node's parent equals itself (the start sentinel). The resulting sequence, which is in reverse order (goal to start), is reversed with `std::reverse` before being returned.

4) *smoothPath()* — *Greedy Shortcutting*: The raw A* path consists of many single-cell steps, producing a jagged trajectory unsuitable for direct robot control. `smoothPath()` applies a greedy shortcutting algorithm:

- 1) Insert the first waypoint and mark it as the *anchor*.
- 2) From the anchor, attempt to connect directly to the *farthest* remaining waypoint via `isLineFree()`. Search proceeds from the last waypoint backward.
- 3) Once a free line-of-sight connection is found, add that waypoint to the pruned path and advance the anchor.
- 4) Repeat until the end of the path is reached.

After pruning, headings are recomputed as the `atan2` direction between consecutive pruned waypoints. The last waypoint inherits the heading of its predecessor to maintain angular continuity.

5) *isLineFree()* — *Line-of-Sight Collision Check*: Given two world-coordinate endpoints (x_0, y_0) and (x_1, y_1) , this function samples points along the connecting line at intervals of $r/2$ (half the grid resolution). At each sample, the world coordinate is converted to a grid cell and two conditions are checked:

- The cell must be within the grid boundaries.
- The distance to the nearest obstacle must be at least 0.20 m.

If any sample fails, the function returns **false**; otherwise it returns **true**. The 0.20m threshold is consistent with the **min_clearance** used during the A* search, ensuring uniform safety margins across the entire pipeline.

II. MAP EXPLORATION

Autonomous exploration enables a robot to navigate and map an unknown environment without prior knowledge of the surroundings. In this project, we implement a frontier-based exploration strategy that allows the MBot robot to progressively explore an unknown map by navigating toward frontier regions. Frontiers are defined as the boundaries between known free space and unexplored regions in the occupancy grid map. By moving toward these boundaries, the robot gradually expands the mapped area and discovers new portions of the environment.

The exploration system is composed of three primary components: a frontier detection module, an exploration management node, and a navigation node responsible for path planning. The frontier detection module identifies potential exploration targets from the map. The exploration node manages the overall exploration behavior and selects new goals. Finally, the navigation node computes safe paths toward these goals using the A* path planning algorithm.

A. System Architecture

The exploration framework is implemented using several ROS2 nodes that communicate through topics and services. The three major components of the system are:

- **frontier_explorer.cpp**: Detects frontier cells from the occupancy grid and selects suitable exploration goals.
- **exploration_node.cpp**: Controls the exploration process, requests frontier goals, and manages path planning.
- **navigation_node.cpp**: Computes collision-free paths using the A* planner and publishes navigation waypoints.

Together, these modules form a complete exploration pipeline. The frontier explorer analyzes the map to determine potential exploration targets. The exploration node selects a goal and requests a path from the navigation node. The navigation node then generates a safe trajectory that the robot follows using its motion controller.

B. Frontier Detection and Goal Selection

The frontier detection algorithm is implemented in **frontier_explorer.cpp**, which defines the **FrontierExplorer** class. The primary function in this class is **selectGoal()**, which determines the next exploration target based on the current robot pose and the obstacle distance grid.

The **selectGoal()** function executes a multi-step pipeline. First, the robot's pose in world coordinates is converted into grid coordinates using the occupancy

grid's resolution and origin. Next, the function calls **detectFrontiers()** to identify raw frontier points in the map. The detected frontier cells are then filtered to ensure they are safe and reachable. After filtering, nearby frontier points are grouped into clusters, and a representative centroid is calculated for each cluster. Finally, the closest valid frontier centroid is selected as the exploration goal.

The overall frontier selection pipeline can be summarized as:

```
selectGoal()
    detectFrontiers()
    clearance filtering
    clusterFrontiers()
    choose closest frontier cluster
```

This pipeline ensures that the robot always selects a safe and meaningful target that expands the explored region.

C. Frontier Cell Detection

The frontier detection process is implemented in the function **detectFrontiers()**. This function scans through the entire occupancy grid and checks each cell to determine whether it qualifies as a frontier.

For each grid cell, the algorithm calls the function **isFrontierCell(x, y, dist_grid)**. A cell is classified as a frontier if it satisfies two conditions:

- 1) The cell itself represents free space in the occupancy grid.
- 2) At least one of its eight neighboring cells is unknown.

The first condition ensures that the robot can physically reach the location, while the second condition indicates that unexplored space exists nearby. When both conditions are satisfied, the cell represents the boundary between explored and unexplored regions.

Once a frontier cell is detected, its grid coordinates are converted to world coordinates using the **gridToWorld()** function. These coordinates are then stored in a list of candidate frontier points.

D. Frontier Safety Filtering

After raw frontier points are detected, the algorithm filters out unsafe frontier locations. This step uses the obstacle distance grid, which stores the distance from each grid cell to the nearest obstacle.

For each frontier candidate, the algorithm checks whether the distance to the nearest obstacle exceeds a predefined safety threshold:

```
dist_grid.getDistance(gx, gy) >= goal_clearance
```

The parameter **goal_clearance** ensures that the robot only navigates to frontier points that have sufficient clearance from obstacles. Without this filtering step, the robot might attempt to move toward frontier points that are too close to walls or obstacles, which could lead to navigation failures or collisions.

E. Frontier Clustering

Raw frontier detection often produces many adjacent frontier cells along the same boundary between explored and unexplored space. Treating each cell as an independent goal would lead to redundant navigation targets and inefficient exploration.

To address this issue, the algorithm groups nearby frontier cells into clusters using the function `clusterFrontiers()`. The clustering process is implemented using a Breadth First Search (BFS) strategy. The algorithm begins by selecting an unvisited frontier point and searching for neighboring frontier points within a specified distance threshold. These neighboring points are added to the same cluster. The search continues until no additional neighbors can be found.

Once all clusters are identified, the centroid of each cluster is computed. The centroid represents the geometric center of the cluster and serves as a single representative exploration target for that region. This approach significantly reduces the number of candidate goals and improves the efficiency of the exploration process.

F. Goal Selection Strategy

After clustering is completed, the algorithm selects the most suitable frontier cluster to explore next. For each cluster centroid, the distance to the robot's current position is calculated using the Euclidean distance formula:

$$\text{distance} = \sqrt{dx^2 + dy^2}$$

Clusters that are too close to the robot are ignored using the parameter `min_frontier_distance`. This prevents the robot from repeatedly selecting frontier targets that are already near its current location.

Among the remaining clusters, the algorithm selects the closest centroid as the next exploration goal. The robot's desired orientation toward the goal is then computed using:

$$\text{theta} = \text{atan2}(dy, dx)$$

The final exploration target is returned as a `Pose2D` goal consisting of the selected position and heading.

G. Exploration Management Node

The high-level exploration behavior is implemented in `exploration_node.cpp`. This node acts as the central controller that coordinates frontier detection and path planning.

The exploration node performs several key tasks:

- Subscribes to the occupancy grid map
- Tracks the robot pose using TF transforms
- Requests frontier goals from the frontier explorer
- Generates paths to exploration targets
- Publishes waypoints to the motion controller

The exploration logic is executed periodically within the `timerCallback()` function. During each iteration, the node first checks whether a valid map has been

received. It then retrieves the current robot pose using the transform system. If the robot has reached the previous goal or no goal is currently active, the node requests a new exploration target by calling `frontier_explorer.selectGoal()`.

If a valid frontier goal is found, the exploration node proceeds to generate a path toward that goal.

H. Path Planning in Exploration

Path generation is performed by the `planPath()` function within the exploration node. This function computes a navigation path from the robot's current position to the selected frontier goal.

The function first verifies that the necessary information is available. It checks whether a recent map has been received, retrieves the robot's pose using the `getRobotPose()` function, and confirms that a valid goal position has been set. If any of these conditions are not satisfied, the function terminates and reports a warning. Once all required information is available, the function calls the A* path planner:

```
astar_planner_.planPath(dist_grid_, robot_pose,
goal, path)
```

The planner uses the obstacle distance grid to compute a collision-free path that maintains a safe distance from obstacles. If a valid path is generated, the resulting waypoints are published for execution and visualization.

I. Navigation and Path Planning

The navigation functionality is implemented in `navigation_node.cpp`, which is responsible for computing the shortest collision-free path between two points.

This node subscribes to several topics, including the occupancy grid map, the robot's pose, and the target goal position. Whenever the map is updated, the node constructs an obstacle distance grid that stores the distance from every cell to the nearest obstacle.

The distance grid plays an important role in ensuring safe navigation. By incorporating obstacle distances into the planning process, the robot can maintain sufficient clearance from walls and obstacles while navigating through the environment.

Once both the start pose and goal pose are available, the node calls the A* planner to compute a path. If planning succeeds, the resulting trajectory is published both as a set of waypoints for the motion controller and as a visualization path for debugging and monitoring purposes.

J. Summary

The frontier-based exploration system enables the MBot robot to autonomously explore unknown environments. By identifying frontier regions, selecting safe exploration goals, and planning collision-free paths, the robot can progressively expand its map of the environment. The integration of frontier detection, exploration management, and A* path planning provides a robust framework for autonomous exploration in unknown spaces.

III. MAP LOCALIZATION WITH UNKNOWN STARTING POSITION

In some scenarios, the robot may be placed on a known map, but without any knowledge of its initial position or orientation. Unlike standard localization, where particles are initialized around a known starting pose, global localization requires the robot to determine its pose anywhere on the map using only sensor observations and motion updates. In this project, we propose a localization strategy based on a particle filter combined with safe exploratory motion to allow the robot to gradually reduce the pose uncertainty.

A. Global Particle Initialization

To handle the unknown starting position, the particle filter must be initialized differently from the normal localization case. Instead of placing particles around a known pose estimate, particles are uniformly distributed across all free cells of the occupancy grid map. Each particle represents a possible hypothesis of the robot's pose, including its position and orientation.

The initialization process first scans the occupancy grid to identify all cells that correspond to free space. A set of particles is then randomly sampled from these free cells. For each particle, the orientation is randomly drawn from the interval $[-\pi, \pi]$. This ensures that the entire feasible configuration space of the robot is represented.

Formally, the initialization process can be described as:

- Identify all grid cells where occupancy value indicates free space.
- Randomly sample particle positions from these free cells.
- Assign a random orientation to each particle.
- Set all particle weights equally.

This uniform initialization allows the robot to consider all possible starting locations on the map.

B. Particle Filter Localization Update

After initialization, the particle filter operates in an iterative loop consisting of motion updates, sensor updates, and resampling steps.

During the motion update step, odometry information is used to update each particle's pose estimate. Noise is added to the motion model to account for uncertainty in wheel encoders and robot movement.

Next, the sensor update step evaluates the likelihood of each particle using Lidar observations. For each particle, the expected laser scan is estimated using the map, and the similarity between the expected scan and the observed scan is calculated. Particles whose predicted measurements match the observed data receive higher weights.

After computing particle weights, a resampling step is performed to generate a new particle set. Particles with higher weights are more likely to be selected, while particles with low likelihood are removed. Over time, this process causes the particle distribution to concentrate near the true robot pose.

C. Safe Motion Without Path Planning

When the robot does not know its location, it cannot safely use a global path planner such as A*. Therefore, a simple motion strategy must be used to allow the robot to move safely while gathering sensor information.

In our approach, we use a basic obstacle avoidance behavior based on LiDAR readings. The robot moves forward when the path ahead is clear, and rotates when an obstacle is detected within a certain distance threshold. This strategy allows the robot to explore its surroundings while avoiding collisions.

The motion strategy can be summarized as follows:

- If the front laser distance is greater than a safety threshold, move forward slowly.
- If an obstacle is detected ahead, rotate in place until a clear direction is found.
- Continue moving while updating the particle filter.

This movement allows the robot to observe new parts of the environment, which helps eliminate incorrect particle hypotheses.

D. Localization Detection

As the robot moves and new sensor data is incorporated, the particle distribution gradually converges toward the true robot pose. To determine when localization has succeeded, the algorithm monitors the spatial variance of the particle set.

Localization is considered successful when the particles become highly concentrated within a small region of the map. This can be detected by computing the variance of particle positions or by checking whether a large proportion of particle weights are concentrated around a single cluster.

Once the variance falls below a predefined threshold, the robot's pose estimate is considered reliable. At this point, the system can switch from the exploratory localization behavior to the normal navigation pipeline.

E. Integration with Navigation System

After successful localization, the estimated robot pose can be published to the rest of the navigation system through the TF transform tree. The robot can then begin using the standard navigation modules implemented in Section 3.3, including the A* path planner and frontier exploration algorithm.

This integration allows the robot to first determine its location in the map and then continue with autonomous exploration and navigation tasks.

F. Algorithm Overview

The overall global localization procedure can be summarized by the following pseudocode:

```
Initialize particles uniformly across free space
while robot is not localized:
```

```
obtain LiDAR scan

motion_update(particles, odometry)

sensor_update(particles, scan, map)

resample_particles(particles)

if particle_variance < threshold:
    localization_success = true
else:
    execute obstacle_avoidance_motion()

switch to normal navigation
```

This strategy enables the robot to gradually reduce pose uncertainty through sensor observations and motion until a reliable localization estimate is obtained.